LAPORAN PRAKTIKUM TEKNOLOGI IoT II

Modul 6

Di susun oleh:

Nama : Virnarokaren

NPM : 25782115

Kelas : TRI-3D

Dosen Pengampu:

Dr. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

Andi Saptono, S.Kom

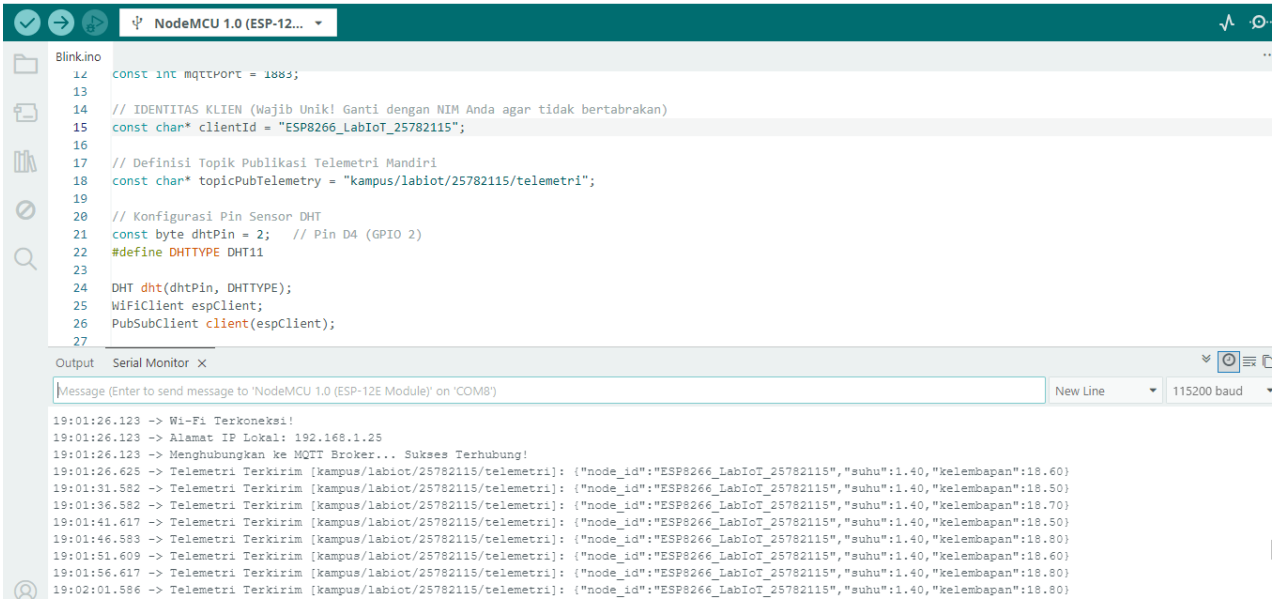
TEKNOLOGI REKAYASA INTERNET

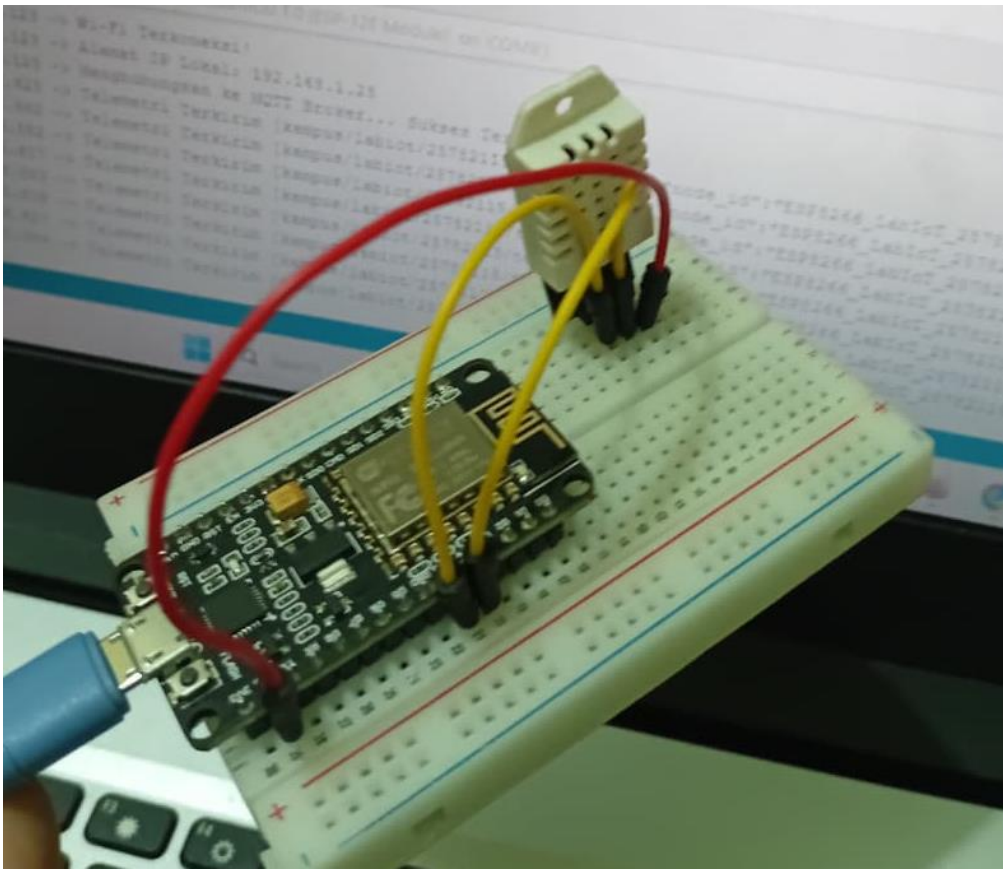
JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

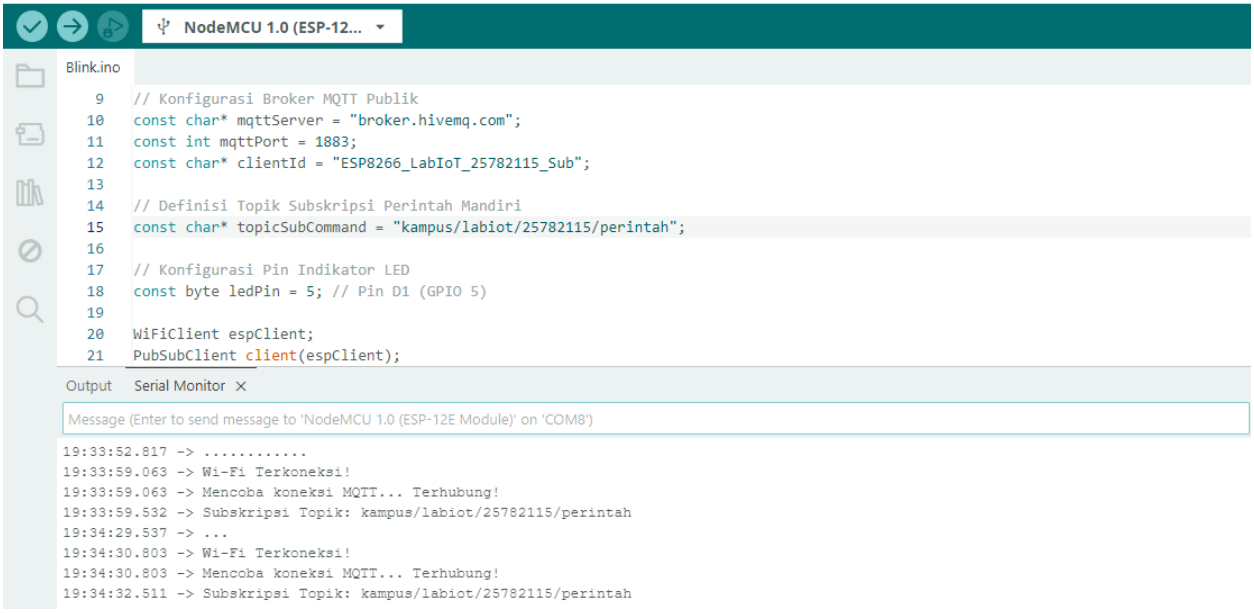
2026/2027

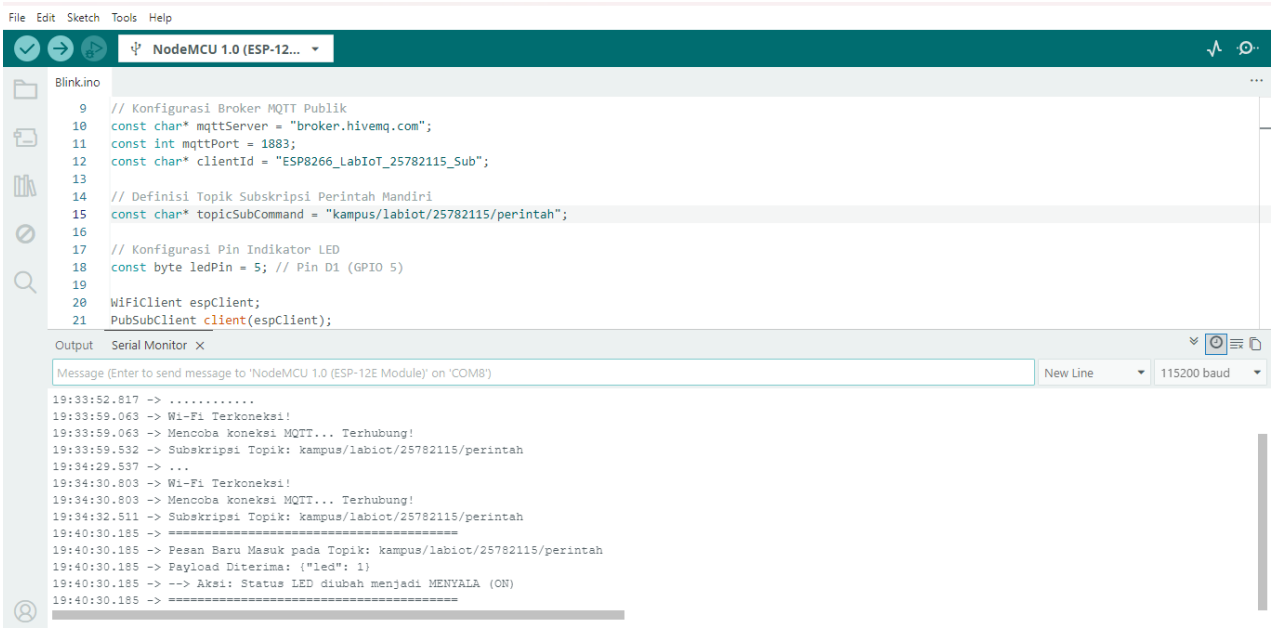
PRAKTIKUM 1

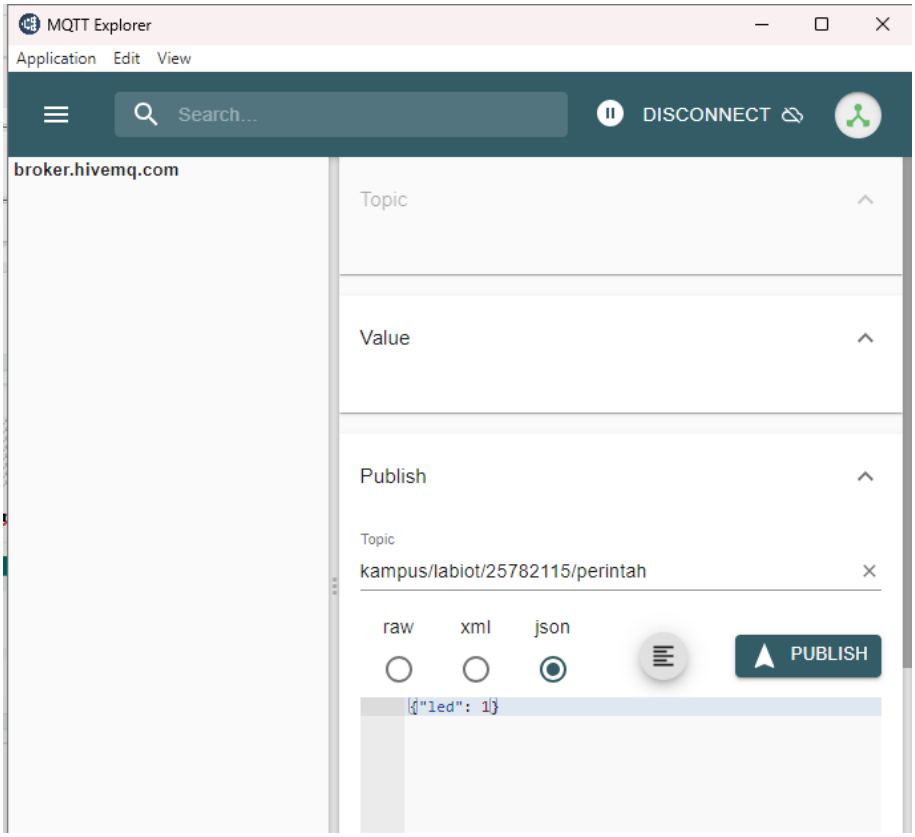

NodeMCU 1.0 (ESP-12...
Blink.ino
const int mqttPort = 1883;
// IDENTITAS KLIEN (Wajib Unik! Ganti dengan NIM Anda agar tidak bertabrakan)
const char* clientId = "ESP8266_LabIoT_25782115";
// Definisi Topik Publikasi Telemetri Mandiri
const char* topicPubTelemetry = "kampus/labiot/25782115/telemetri";
// Konfigurasi Pin Sensor DHT
const byte dhtPin = 2; // Pin D4 (GPIO 2)
#define DHTTYPE DHT11
DHT dht(dhtPin, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);
Output
Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM8')
New Line
115200 baud
19:01:26.123 -> Wi-Fi Terkoneksi!
19:01:26.123 -> Alamat IP Lokal: 192.168.1.25
19:01:26.123 -> Menghubungkan ke MQTT Broker... Sukses Terhubung!
19:01:26.625 -> Telemetri Terkirim [kampus/labiot/25782115/telemetri]: {"node_id":"ESP8266_LabIoT_25782115","suhu":1.40,"kelembapan":18.60}
19:01:31.582 -> Telemetri Terkirim [kampus/labiot/25782115/telemetri]: {"node_id":"ESP8266_LabIoT_25782115","suhu":1.40,"kelembapan":18.50}
19:01:36.582 -> Telemetri Terkirim [kampus/labiot/25782115/telemetri]: {"node_id":"ESP8266_LabIoT_25782115","suhu":1.40,"kelembapan":18.70}
19:01:41.617 -> Telemetri Terkirim [kampus/labiot/25782115/telemetri]: {"node_id":"ESP8266_LabIoT_25782115","suhu":1.40,"kelembapan":18.50}
19:01:46.583 -> Telemetri Terkirim [kampus/labiot/25782115/telemetri]: {"node_id":"ESP8266_LabIoT_25782115","suhu":1.40,"kelembapan":18.80}
19:01:51.609 -> Telemetri Terkirim [kampus/labiot/25782115/telemetri]: {"node_id":"ESP8266_LabIoT_25782115","suhu":1.40,"kelembapan":18.60}
19:01:56.617 -> Telemetri Terkirim [kampus/labiot/25782115/telemetri]: {"node_id":"ESP8266_LabIoT_25782115","suhu":1.40,"kelembapan":18.80}
19:02:01.586 -> Telemetri Terkirim [kampus/labiot/25782115/telemetri]: {"node_id":"ESP8266_LabIoT_25782115","suhu":1.40,"kelembapan":18.80}

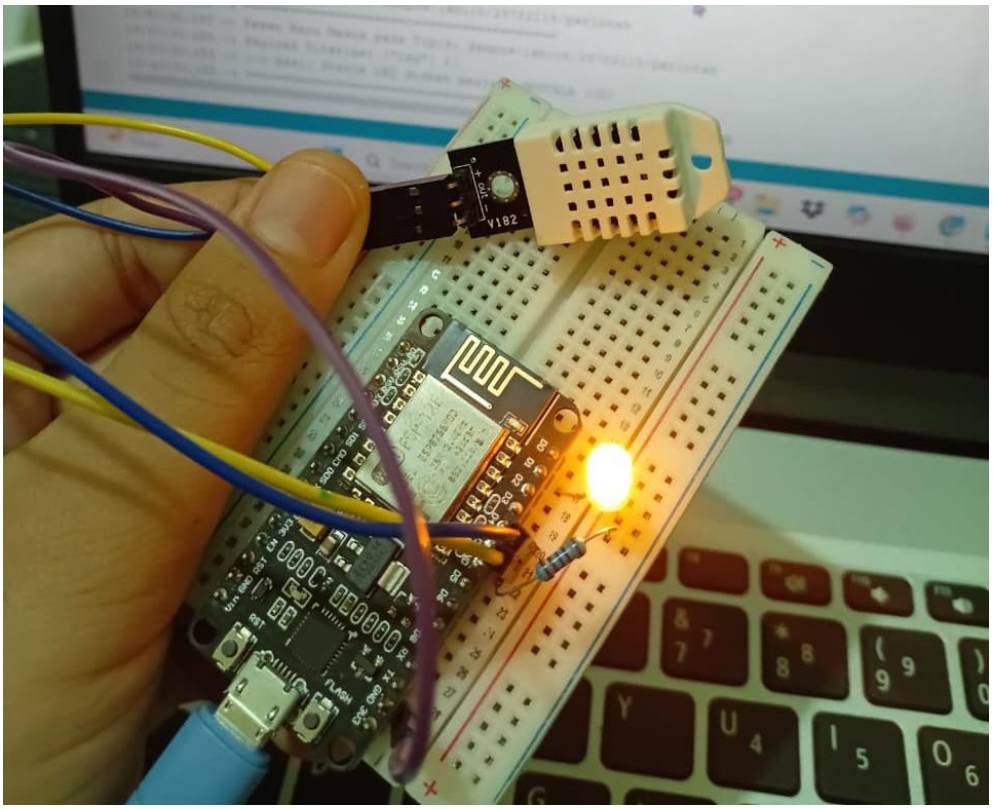


6.5.2 Praktikum 2: Subskripsi dan Kontrol Indikator LED Asinkron

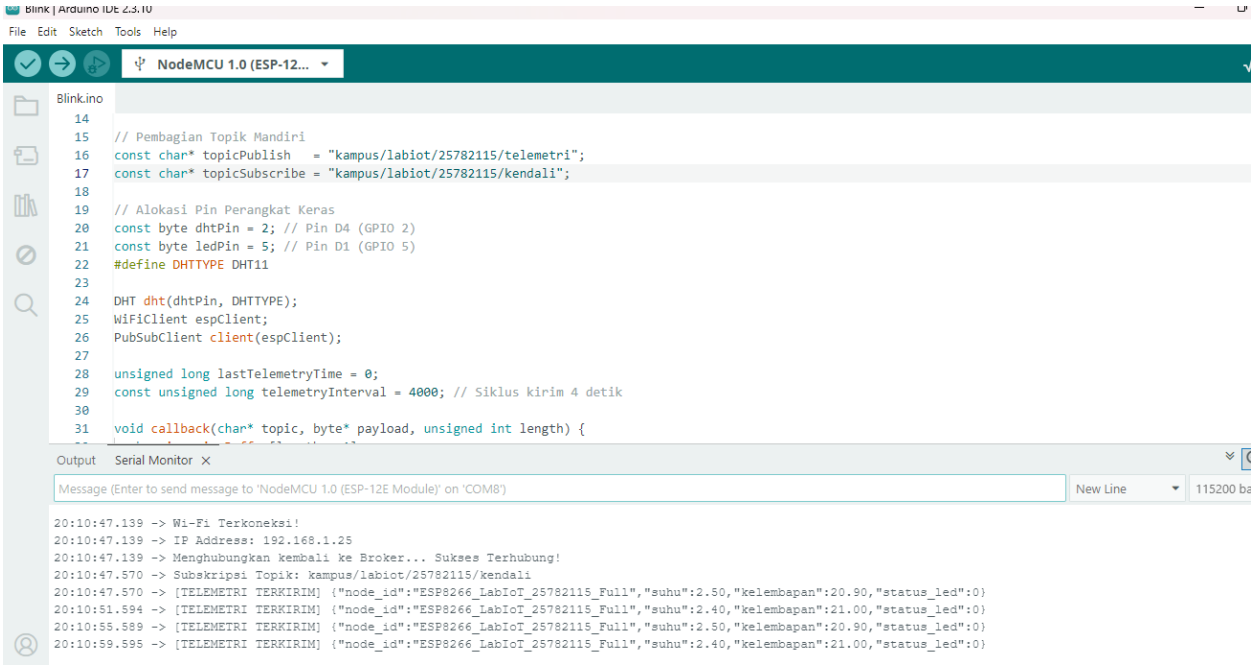




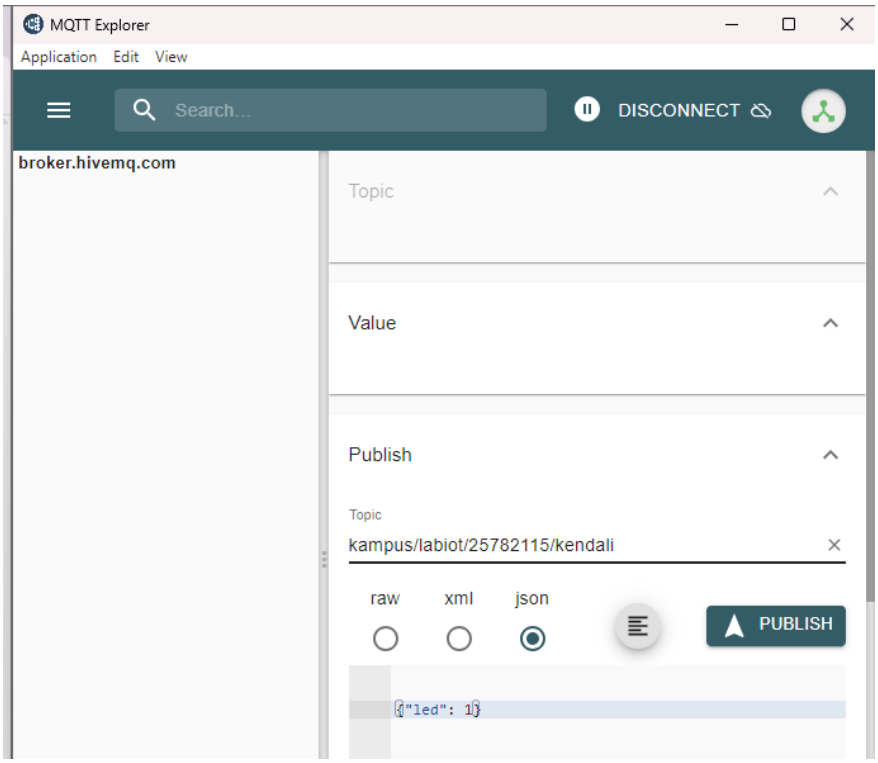

MQTT Explorer
Application Edit View
Search...
DISCONNECT
broker.hivemq.com
Topic
Value
Publish
Topic
kampus/labiot/25782115/perintah
raw
xml
json
PUBLISH
{"led": 1}

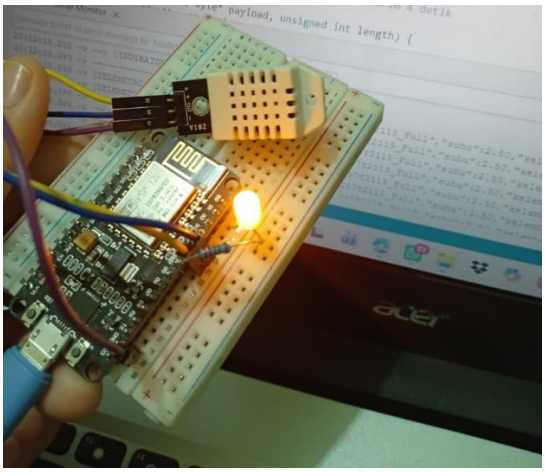


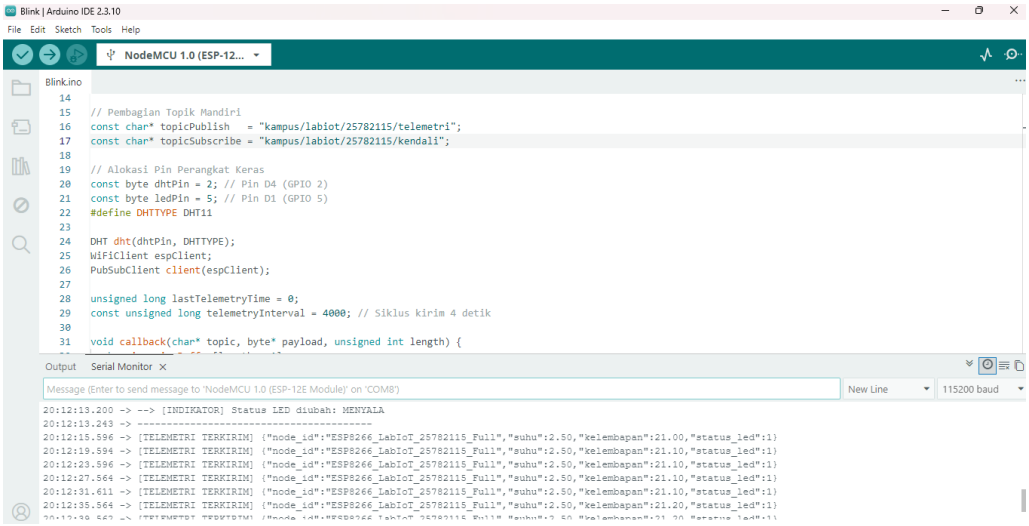
6.5.3 Praktikum 3: Sistem Terpadu Dua Arah (Full Duplex Telemetry & Control)



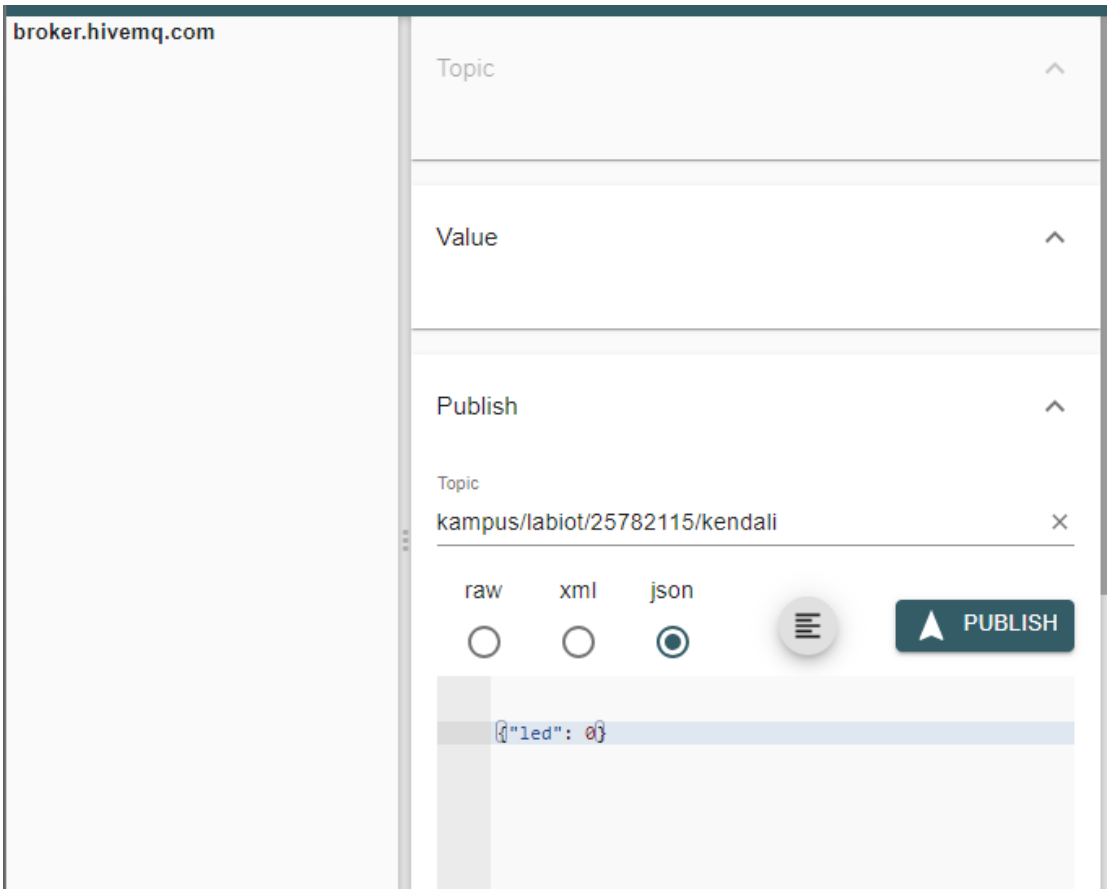
Perintah on

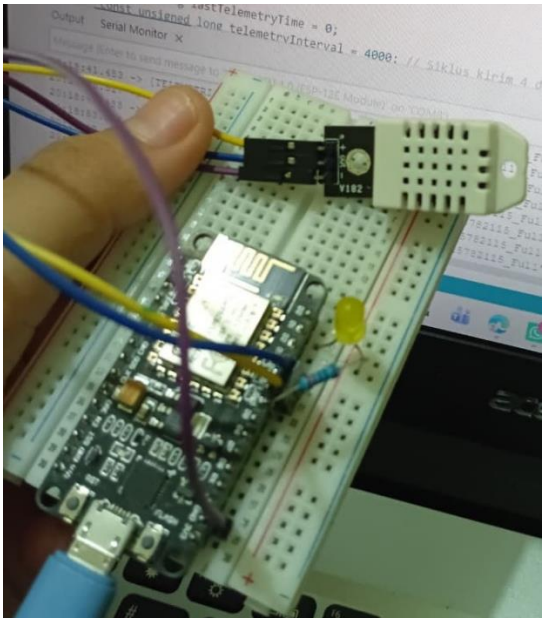


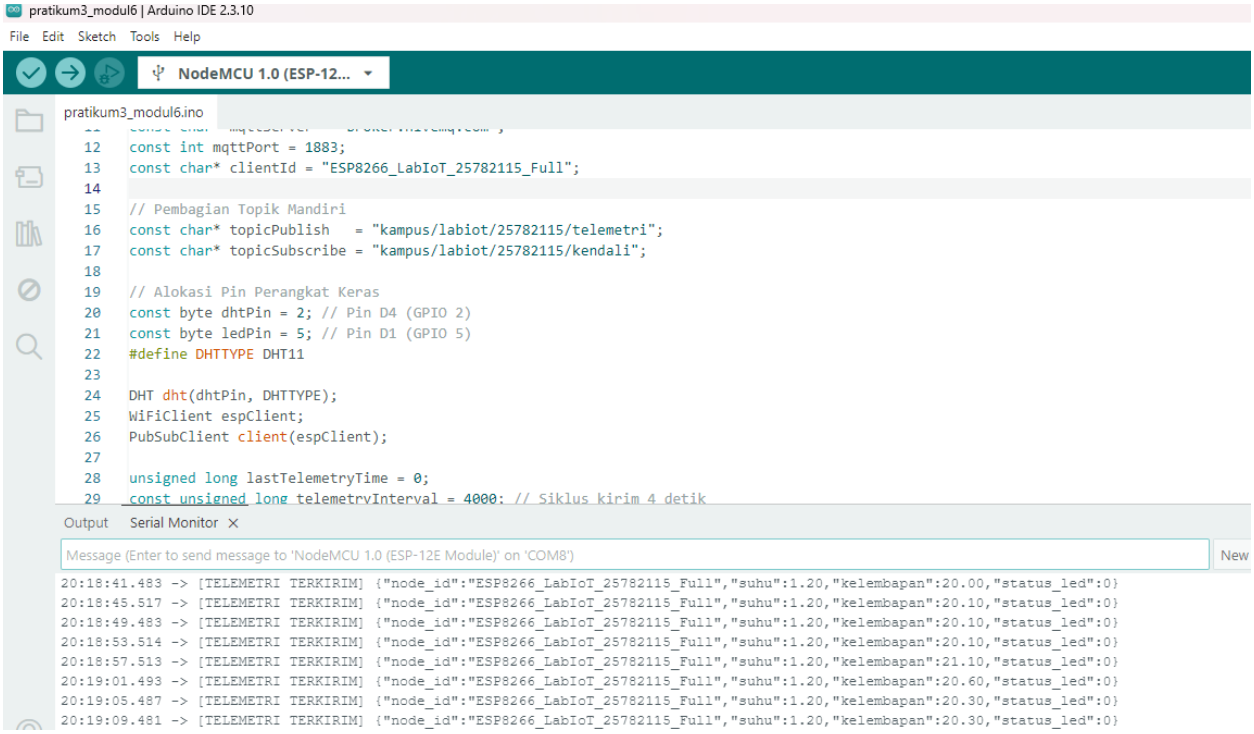




Perintah off


broker.hivemq.com
Topic
Value
Publish
Topic
kampus/labiot/25782115/kendali
raw
xml
json
PUBLISH
{"led": 0}




pratikum3_modul6 | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...
pratikum3_modul6.ino
12 const int mqttPort = 1883;
13 const char* clientId = "ESP8266_LabIoT_25782115_Full";
14
15 // Pembagian Topik Mandiri
16 const char* topicPublish = "kampus/labiot/25782115/telemetri";
17 const char* topicSubscribe = "kampus/labiot/25782115/kendali";
18
19 // Alokasi Pin Perangkat Keras
20 const byte dhtPin = 2; // Pin D4 (GPIO 2)
21 const byte ledPin = 5; // Pin D1 (GPIO 5)
22 #define DHTTYPE DHT11
23
24 DHT dht(dhtPin, DHTTYPE);
25 WiFiClient espClient;
26 PubSubClient client(espClient);
27
28 unsigned long lastTelemetryTime = 0;
29 const unsigned long telemetryInterval = 4000; // Siklus kirim 4 detik
Output Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM8')
New
20:18:41.483 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782115_Full","suhu":1.20,"kelembapan":20.00,"status_led":0}
20:18:45.517 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782115_Full","suhu":1.20,"kelembapan":20.10,"status_led":0}
20:18:49.483 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782115_Full","suhu":1.20,"kelembapan":20.10,"status_led":0}
20:18:53.514 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782115_Full","suhu":1.20,"kelembapan":20.10,"status_led":0}
20:18:57.513 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782115_Full","suhu":1.20,"kelembapan":21.10,"status_led":0}
20:19:01.493 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782115_Full","suhu":1.20,"kelembapan":20.60,"status_led":0}
20:19:05.487 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782115_Full","suhu":1.20,"kelembapan":20.30,"status_led":0}
20:19:09.481 -> [TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782115_Full","suhu":1.20,"kelembapan":20.30,"status_led":0}

Latihan 1 (Analisis Arsitektur & Perbandingan Protokol):
Pada Modul 3, kita menggunakan protokol HTTP untuk membaca dan mengontrol perangkat. Jelaskan secara mendalam mengapa protokol MQTT jauh lebih unggul daripada HTTP dalam hal:

1. Efisiensi penggunaan bandwidth pada paket data seluler

MQTT lebih efisien dalam penggunaan bandwidth dibandingkan HTTP karena memiliki ukuran header paket yang lebih kecil, yaitu minimal 2 byte. Sementara itu, HTTP memiliki header yang lebih besar karena membawa informasi tambahan dalam setiap permintaan dan respons. Oleh karena itu, MQTT dapat mengurangi penggunaan kuota internet ketika perangkat IoT mengirimkan data sensor secara berkala, sehingga cocok digunakan pada jaringan seluler dengan bandwidth terbatas.

2. Beban komputasi mikrokontroler (CPU overhead) saat menangani komunikasi dua arah

MQTT dapat mengurangi beban komunikasi pada mikrokontroler karena menggunakan mekanisme Publish-Subscribe melalui MQTT Broker. Dengan mekanisme ini, perangkat dapat mengirim dan menerima pesan melalui koneksi yang dipertahankan tanpa harus membuat permintaan HTTP berulang kali. Struktur pesan MQTT yang sederhana juga membantu mengurangi pemrosesan data komunikasi, sehingga mikrokontroler dapat menjalankan tugas lain, seperti membaca sensor dan mengendalikan LED.

3. Kemampuan pengiriman data secara simultan ke banyak perangkat penerima (1-to-N distribution)

MQTT memudahkan pengiriman data ke banyak perangkat melalui mekanisme Publish-Subscribe. Pengirim cukup memublikasikan pesan ke satu topik, kemudian MQTT Broker meneruskan pesan tersebut kepada semua perangkat yang berlangganan pada topik tersebut. Dengan cara ini, pengirim tidak perlu mengirim data satu per satu kepada setiap penerima, sehingga distribusi data menjadi lebih efisien dan cocok digunakan pada sistem IoT yang memiliki banyak perangkat.

Latihan 2 (Analisis Hierarki Topik & Karakter Wildcard):
Sebuah pabrik manufaktur memiliki 3 lini produksi (*line1*, *line2*, *line3*), di mana setiap lini memiliki 4 mesin (*m1*, *m2*, *m3*, *m4*), dan setiap mesin melaporkan metrik suhu, getaran, dan status. Struktur topiknya adalah pabrik/<line>/<mesin>/<sensor>. Tentukan ekspresi topik subskripsi (*subscription topic*) yang tepat menggunakan karakter *wildcard* (+ atau #) untuk:

1. Memantau parameter suhu dari seluruh mesin di semua lini produksi.

Ekspresi topik subskripsi yang digunakan adalah pabrik/+/+/suhu. Karakter wildcard + digunakan untuk menggantikan satu tingkat topik, yaitu nama lini produksi dan nama mesin. Dengan demikian, topik ini dapat menerima data suhu dari seluruh mesin di semua lini produksi.

2. Memantau seluruh parameter sensor khusus untuk mesin m1 di lini line2.

Ekspresi topik subskripsi yang digunakan adalah pabrik/line2/m1/+. Karakter wildcard + digunakan untuk menggantikan satu tingkat topik terakhir, yaitu jenis sensor. Dengan demikian, topik ini dapat menerima seluruh data sensor, seperti suhu, getaran, dan status dari mesin m1 di lini line2.

3. Memantau seluruh lalu lintas data dari seluruh lini dan seluruh mesin di pabrik.

Ekspresi topik subskripsi yang digunakan adalah pabrik/#. Karakter wildcard # digunakan untuk mencocokkan semua tingkat topik setelah pabrik/. Dengan demikian, seluruh data dari semua lini produksi, semua mesin, dan semua jenis sensor dapat diterima.

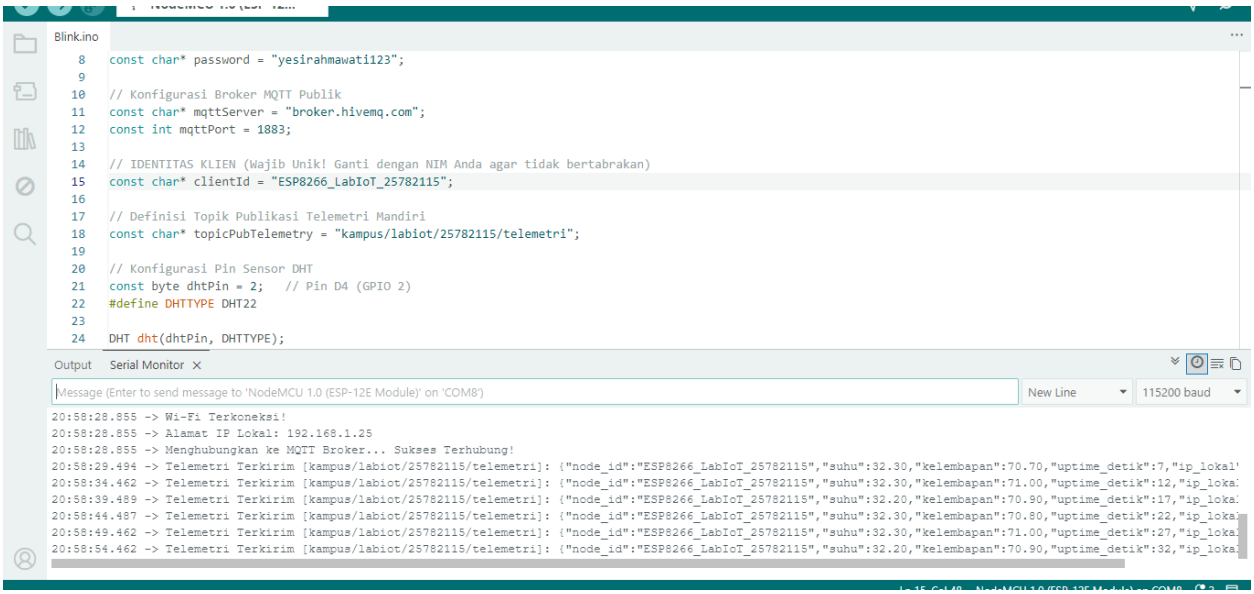
Latihan 3 (Implementasi Serialisasi Data Dinamis):
Modifikasi blok penyusunan JSON pada Praktikum 1 agar paket telemetri menyertakan dua parameter tambahan berikut:

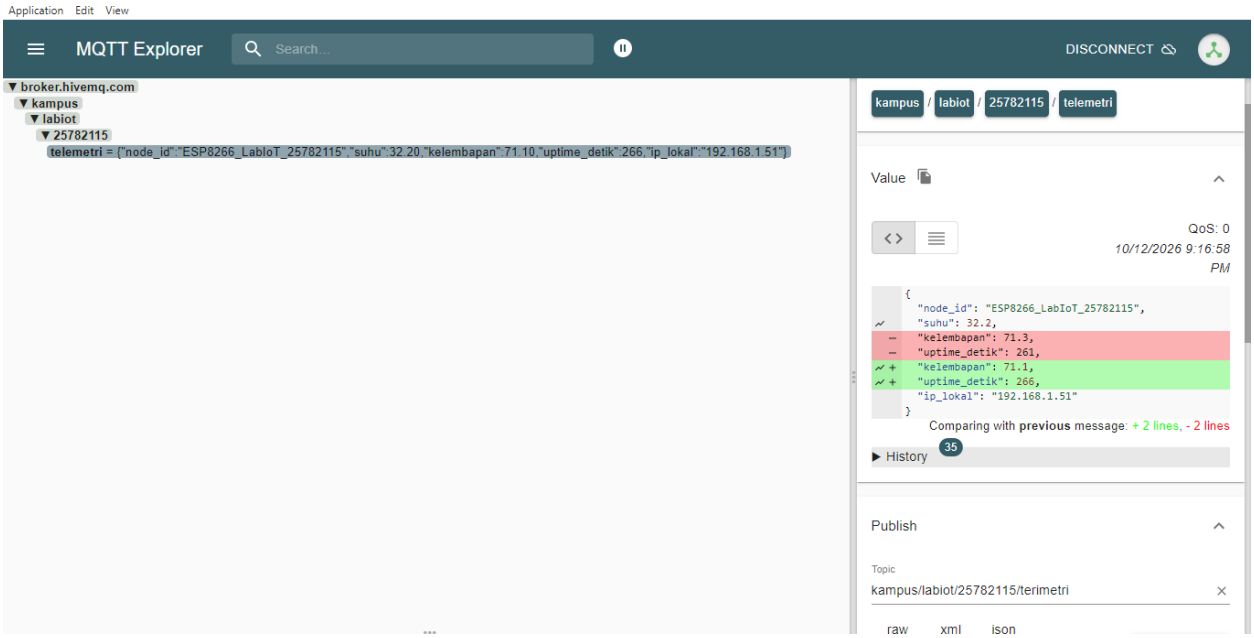
```
// Alokasi dokumen JSON statis
StaticJsonDocument<256> doc;
doc["node_id"] = clientId;
doc["suhu"] = serialized(String(t, 2));        // Format desi
doc["kelembapan"] = serialized(String(h, 2));

doc["uptime_detik"] = millis() / 1000;
doc["ip_lokal"] = WiFi.localIP().toString();


// Serialisasi JSON ke dalam array buffer karakter
char jsonBuffer[256];
serializeJson(doc, jsonBuffer);
```

Pada Latihan 3, saya menambahkan dua parameter baru pada bagian pembentukan JSON dalam program Praktikum 1. Parameter pertama adalah uptime_detik, yang digunakan untuk mengetahui lama waktu NodeMCU aktif dalam satuan detik dengan menggunakan fungsi millis() / 1000. Parameter kedua adalah ip_lokal, yang digunakan untuk menampilkan alamat IP lokal NodeMCU dengan fungsi WiFi.localIP().toString(). Kedua parameter tersebut ditambahkan sebelum proses serialisasi JSON agar informasi waktu aktif dan alamat IP ikut dimasukkan ke dalam data telemetri yang dikirim melalui MQTT.





Latihan 4 (Analisis Mekanisme Reconnect & Loop Blocking):
Pada Praktikum 3, di dalam fungsi loop() terdapat pemanggilan rutin client.loop().

1. Konsekuensi penggunaan delay(6000) terhadap responsivitas callback

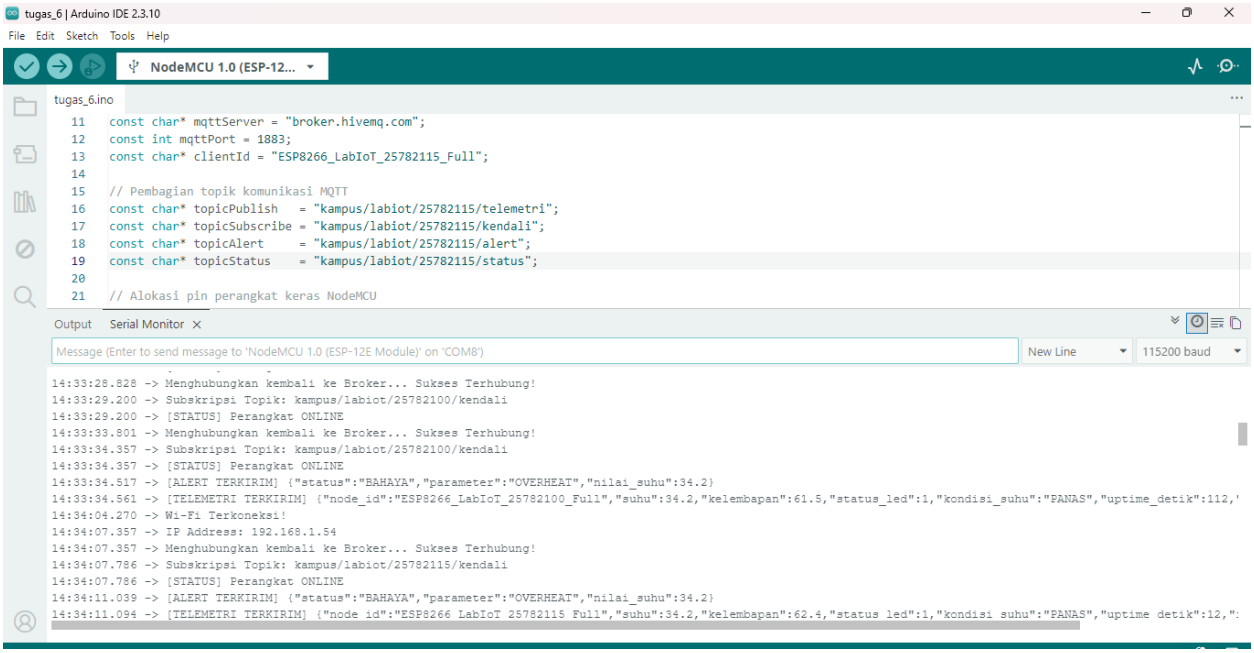
Jika instruksi delay(6000) ditambahkan ke dalam fungsi loop(), program akan berhenti sementara selama 6 detik. Selama waktu tersebut, fungsi client.loop() tidak dapat dijalankan secara rutin sehingga penerimaan dan pemrosesan pesan MQTT, termasuk callback untuk perintah LED,

menjadi terlambat. Akibatnya, respons perangkat terhadap perintah dari MQTT Explorer menjadi kurang responsif.

2. Cara kerja fungsi reconnect() dalam memulihkan koneksi

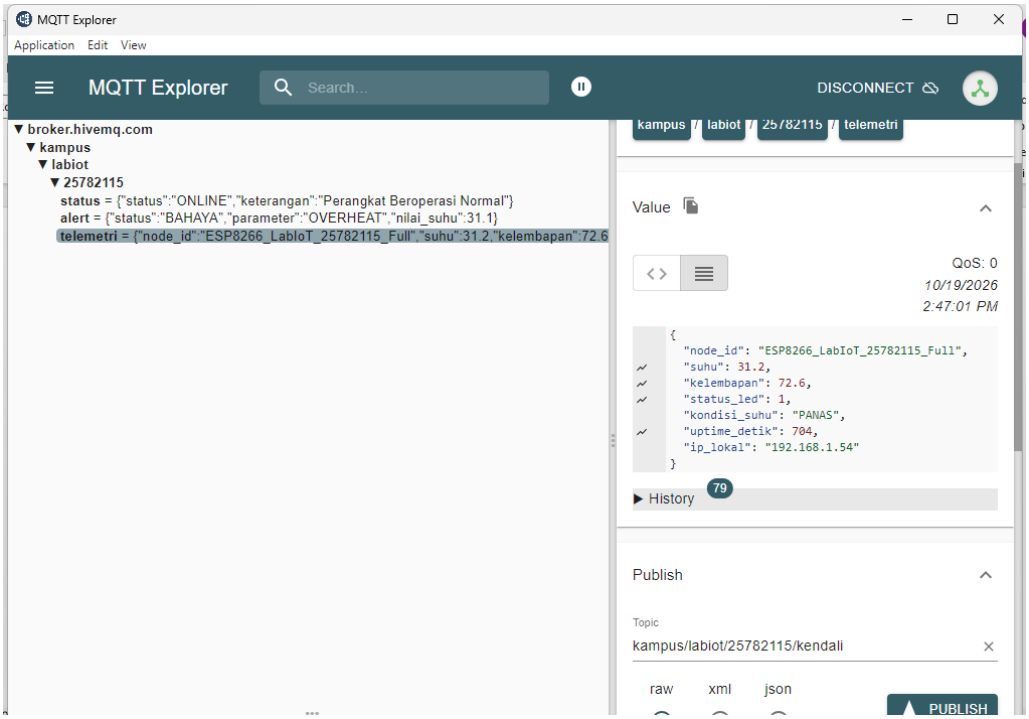
Fungsi reconnect() digunakan untuk mencoba menghubungkan kembali perangkat ke broker MQTT ketika koneksi terputus. Program akan memeriksa status koneksi dan mencoba melakukan koneksi ulang menggunakan client.connect(). Jika koneksi berhasil, perangkat akan berlangganan kembali ke topik kendali menggunakan client.subscribe(), sehingga dapat menerima perintah MQTT seperti sebelumnya. Jika koneksi belum berhasil, program akan mencoba kembali sampai koneksi pulih**.**

6.7 Tugas: Sistem Telemetri & Kendali Closed-Loop Industri

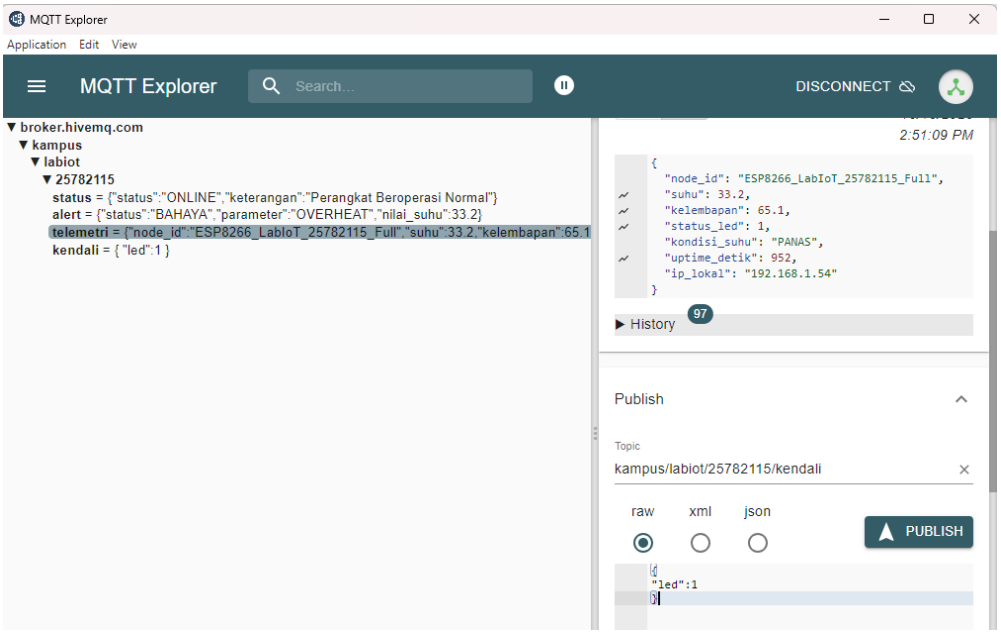


Hasil Output MQTT:

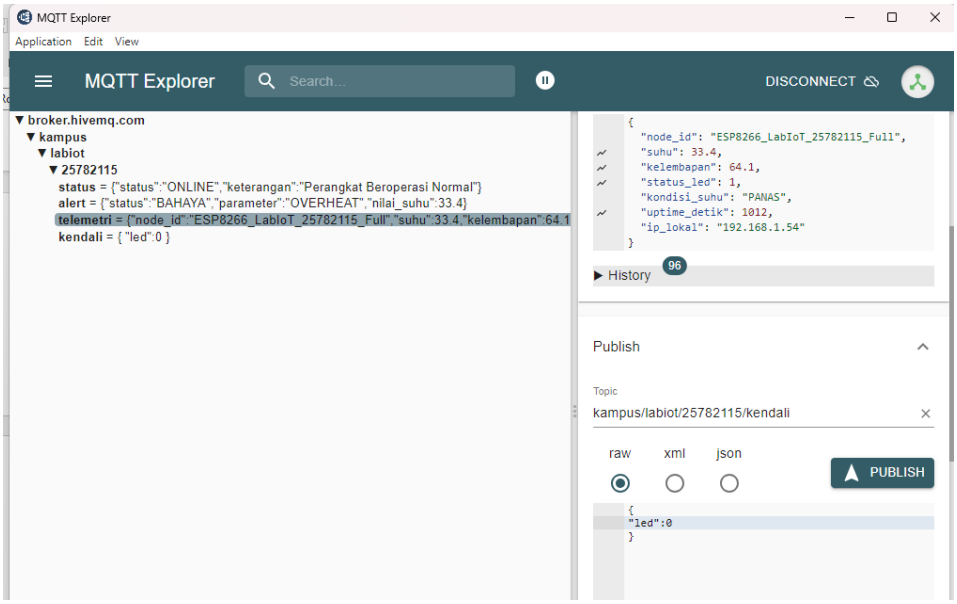
Topic telemetri;



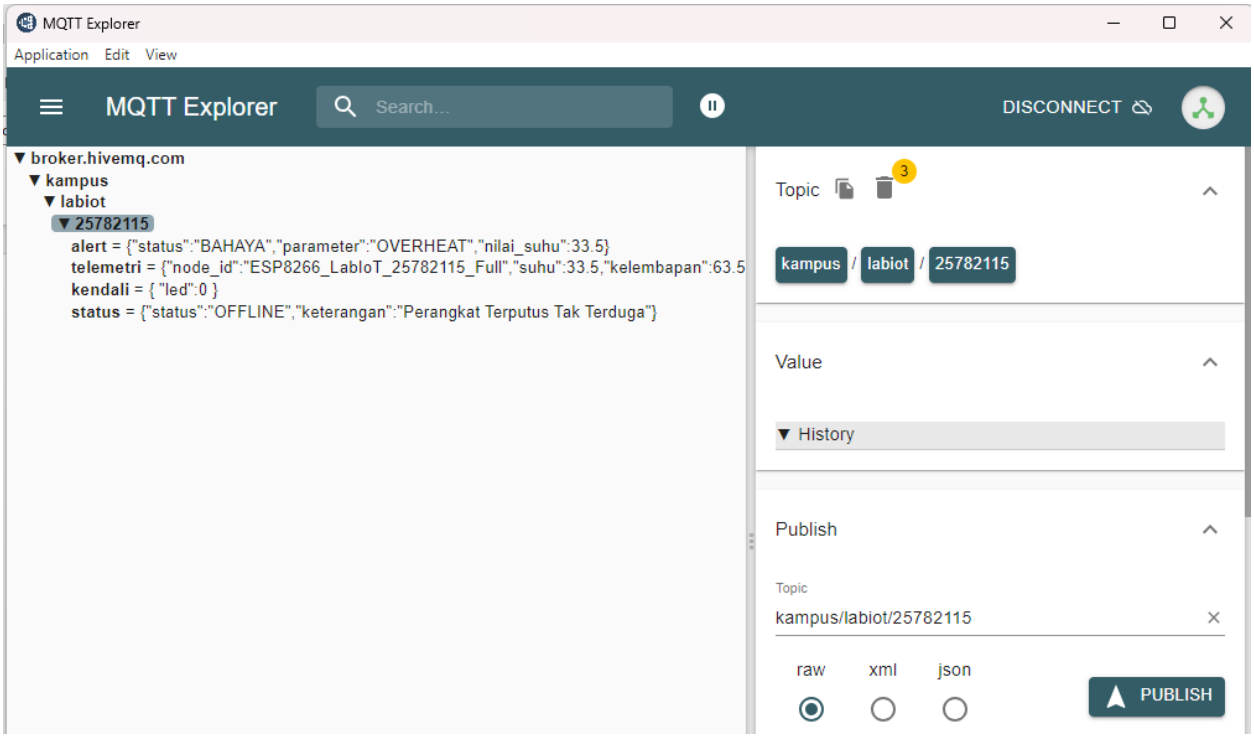
Jika on

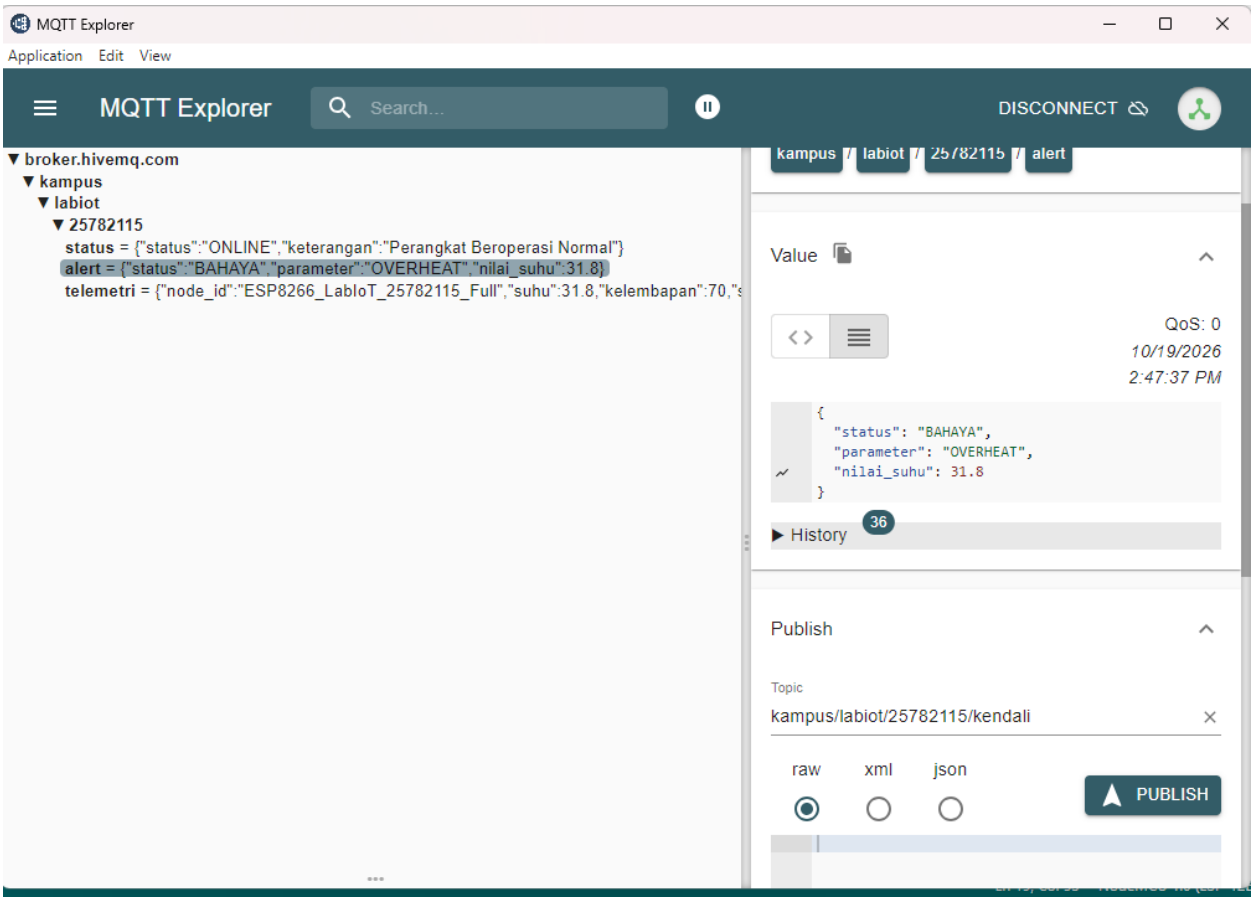


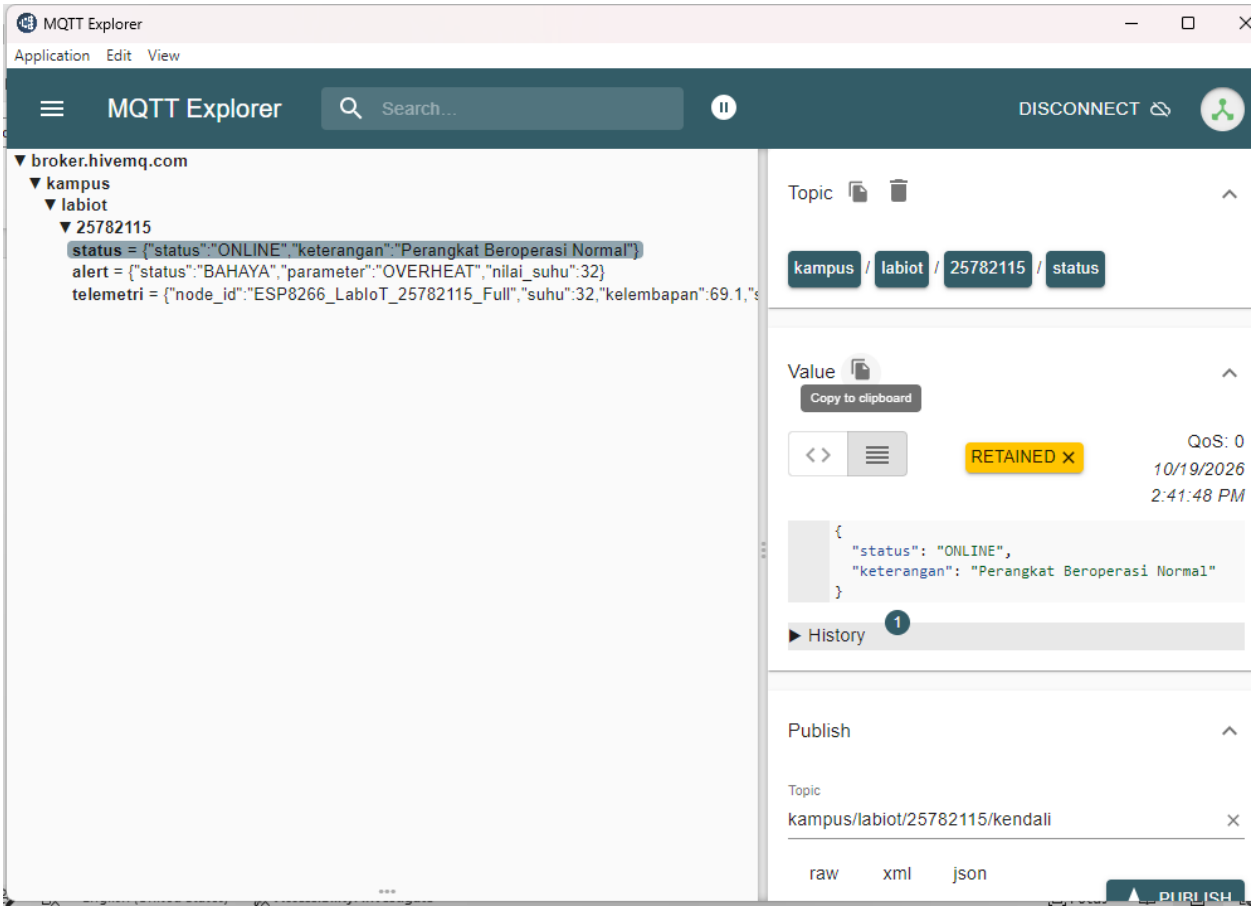
Jika off

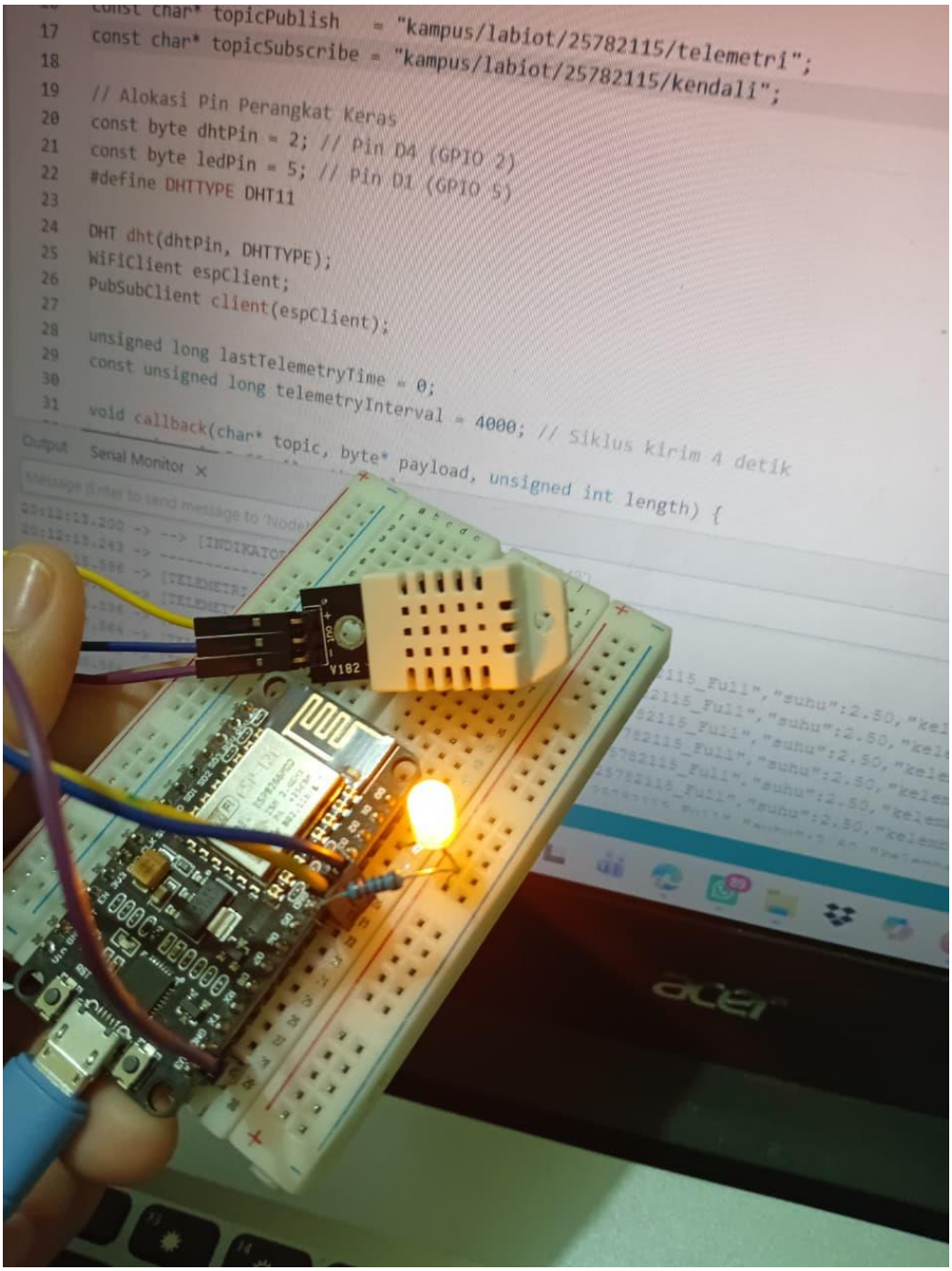


Jika di cabut tiba tiba




MQTT Explorer
Application Edit View
MQTT Explorer
Search...
DISCONNECT
broker.hivemq.com
kampus
labiot
25782115
status = {"status":"ONLINE","keterangan":"Perangkat Beroperasi Normal"}
alert = {"status":"BAHAYA","parameter":"OVERHEAT","nilai_suhu":31.8}
telemetri = {"node_id":"ESP8266_LabIoT_25782115_Full","suhu":31.8,"kelembapan":70,"
kampus / labiot / 25782115 / alert
Value
QoS: 0
10/19/2026
2:47:37 PM
{
"status": "BAHAYA",
"parameter": "OVERHEAT",
"nilai_suhu": 31.8
}
History
36
Publish
Topic
kampus/labiot/25782115/kendali
raw
xml
json
PUBLISH


MQTT Explorer
Application Edit View
MQTT Explorer
Search...
DISCONNECT
broker.hivemq.com
kampus
labiot
25782115
status = {"status":"ONLINE","keterangan":"Perangkat Beroperasi Normal"}
alert = {"status":"BAHAYA","parameter":"OVERHEAT","nilai_suhu":32}
telemetri = {"node_id":"ESP8266_LabIoT_25782115_Full","suhu":32,"kelembapan":69.1,"
Topic
kampus / labiot / 25782115 / status
Value
Copy to clipboard
RETAINED
QoS: 0
10/19/2026
2:41:48 PM
{
"status": "ONLINE",
"keterangan": "Perangkat Beroperasi Normal"
}
History
1
Publish
Topic
kampus/labiot/25782115/kendali
raw
xml
json
PUBLISH



Kesimpulan

Berdasarkan praktikum yang telah dilakukan, sistem pemantauan suhu dan kelembapan berhasil dibuat dengan memanfaatkan NodeMCU ESP8266, sensor DHT22, dan protokol MQTT untuk mengirimkan data secara berkala.

Sistem ini memiliki beberapa fungsi utama, yaitu:

- **Pemantauan Kondisi Ruangan: Sensor DHT22 membaca suhu dan kelembapan ruangan. Jika suhu berada pada 31°C atau kurang, kondisi dikategorikan NORMAL, sedangkan suhu di atas 31°C dikategorikan PANAS.**
- **Peringatan Suhu Otomatis: Ketika suhu melebihi batas yang ditentukan, sistem akan mengaktifkan LED sebagai tanda peringatan. Pada kondisi bahaya, LED tetap menyala sampai kondisi suhu kembali aman sehingga peringatan tidak dapat dinonaktifkan secara manual selama kondisi panas berlangsung.**
- **Pengiriman Data dan Pemantauan Status: Informasi sensor dikirim melalui MQTT ke sistem pemantauan. Fitur Last Will and Testament (LWT) juga digunakan untuk memberikan informasi ketika perangkat berstatus ONLINE atau mengalami pemutusan koneksi dengan status OFFLINE.**
- **Komunikasi yang Responsif: Program menggunakan mekanisme tanpa jeda panjang dari delay() agar proses pembacaan sensor, pengiriman data, dan penerimaan perintah MQTT dapat berjalan lebih responsif.**

Dengan demikian, praktikum ini memberikan pemahaman mengenai penerapan Internet of Things (IoT) untuk memantau kondisi lingkungan, mengirimkan data secara berkala, serta memberikan peringatan otomatis berdasarkan perubahan suhu.